CEG 3156: Computer Systems Design (Winter 2026)
Prof. Rami Abielmona
Laboratory #3: Pipelined Processor

March 3, 2026

Contents

1 Objective **2**

2 Pre-Lab **2**

3 Introduction to Pipelining **2**

4 Pipelining Details **3**
- 4.1 Pipelined Processor Realization 4
- 4.2 Pipeline Hazards 6

5 Laboratory **8**
- 5.1 Memory Blocks 9
- 5.2 Verification: Running a Benchmark Program 12
- 5.3 Bonus: 32-bit Multiprocessor Design 13
- 5.4 Bonus: Exception Handling 13
- 5.5 Bonus: Board Demonstration 14

6 Design Restrictions **14**

7 Report Reminders **15**

8 Acknowledgements **15**

1 Objective

The objective of this laboratory is to design and build a pipelined RISC processor in VHDL.

Upon completion, the student must be able to:

- Design, realize and test a pipelined processor;
- Demonstrate an understanding for pipelining concepts, including hazards, in the context of RISC processors and instruction set architectures.

2 Pre-Lab

Design the hazard detection and forwarding units discussed in this laboratory. Show all your work, including the internals of your design, such as the truth table, optimization and Boolean realization of the circuits. Demonstrate the correct operation of the circuit by analyzing a sample scenario. Present and explain your work to the TA.

3 Introduction to Pipelining

In order to increase processor performance, a designer does not have control over many factors. Two of these rare factors are parallelism and **pipelining**. Both aim to increase the number of supported concurrent operations on the processor. Parallelism involves multiple datapaths capable of executing multiple instructions simultaneously, while pipelining involves a single datapath capable of executing multiple instructions simultaneously. The difference is that pipelining achieves a clock per instruction (CPI) ratio of 1, while parallelism achieves ratios greater than 1, with a tradeoff in area.

We will be looking at an implementation of a pipelined processor using our previously designed, realized and tested single-cycle processor. The latter had an associated hardwired control unit, able to decode and execute on a datapath, various different instructions, mainly R-type, I-type and J-type instructions.

Our implementation will support the same instructions as our hardwired control units. It will not include all integer operations supported by the MIPS ISA (e.g. multiply and divide), nor will it support floating-point operations, also supported by the MIPS processor. However, the design will be modular and expandable in order to support additional operations if the need arises.

Pipelining is the process by which instructions are parallelized over several overlapping stages of execution, in order to maximize datapath efficiency. It is used in modern advanced computer architectures in order to squeeze the most performance from the existing silicon. A real-life analogy involves a car wash process. The eventual goals of the process is to wash any car that enters the car wash, and to simultaneously wash more than 1 car. To achieve those goals, the car wash process is broken down into 5 intermediary stages:

- Hose down the surface dirt off the car
- Spray entire car with soap
- Clean the car with water and powerful brushes
- Rinse the car down
- Dry the car

The concepts introduced by breaking the car wash process into 5 stages, and allowing as many as 5 cars into the car wash at any one time are very important. Firstly, note that new inputs (i.e. dirty cars) are accepted at one end before previously accepted inputs appear as outputs (i.e. clean cars) at the other end. Secondly, the number of operations performed per second is increased, even though the elapsed time needed to perform any one operation (i.e. washing a car) remains the same.

Let us now review the datapath that we've built in the previous lab, mainly the single-cycle MIPS processor. The resultant is shown in figure 10. This processor had two control units, the ALU and the main control units. Those worked in tandem to provide all the control signals required to execute the instructions on the datapath. This laboratory involves pipelining that processor, through its division into 5 stages of execution, as well as the addition of pipeline registers containing both data and control signals.

4 Pipelining Details

Let us now introduce the pipeline that we're working with, basically, the 5-stage MIPS instruction pipeline consisting of the following steps (see figures 1 and 2):

- Fetch instruction from memory (IF)
- Read registers while decoding the instruction (ID)
- Execute the operation or calculate an address (EX)
- Access an operand in data memory (MEM)
- Write the result into a register (WB)

Now, if we were to allow more than one instruction to enter the datapath, ignoring any synchronization that has to occur to allow for the in-step execution of multiple instructions on the same datapath, we can ideally have an instruction throughput of one per clock cycle. It is said that an *instruction is issued* every clock cycle. Refer to figure 3 to see how multiple instructions can execute on the same datapath over time. Note that although the *pipeline latency* is 5 clock cycles (i.e. the time it takes for the first instruction to be issued is 5 clock cycles), the pipeline throughput after that is 1 instruction per clock cycle. Also note that each instruction still requires 5 clock cycles to complete execution.

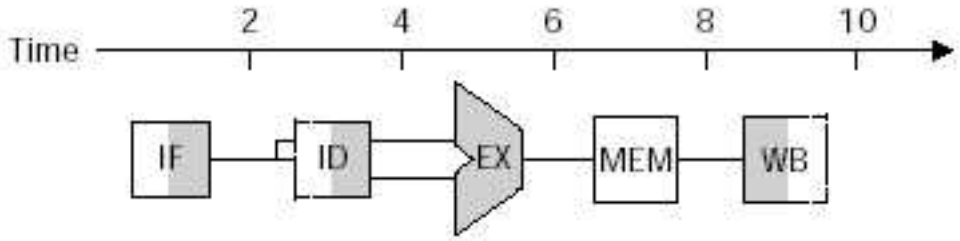

Figure 1: 5-stage MIPS processor

Moreover, in order to convert our single-cycle processor, we will need to break it up into 5 stages, and add what are termed as *pipeline registers* which are analogous to the intermediary registers added when the single-cycle processor was turned into a multicycle one. The functionality of these pipeline registers includes the storage of control and/or data that will be utilized later on in the pipeline. It is easy to note that only 4 of these pipeline registers are required, as the last stage (WB) already has a register (i.e. the register file) that it updates at the end of the clock cycle (see figure 4).

We take a brief break to note that RISC processors, in particular, are designed to take full advantage of pipelining concepts due to their:

Fixed-length instruction format Allows us to synchronize the first two stages of the pipeline: fetching the instruction and decoding it;

Instructions using the same number of execute clock cycles Allows us to normalize instruction throughput;

Instructions using the same resources in each execute clock cycle Allows us to execute all instructions on the same datapath, which is an inherent requirement to pipelining;

Few instruction formats Allows us to symmetrically decode each instruction (e.g. the source register field is located in the same place in each instruction);

Memory operands only appearing in loads and stores Allows us to calculate a memory address in one stage, and access that memory location in the next stage; and

Memory-aligned operands Allows us to transfer data between processor and memory in a single pipeline stage.

4.1 Pipelined Processor Realization

So far, we have avoided discussing the actual synchronization that has to occur in order to ensure the proper operation of the pipeline. Introducing the control

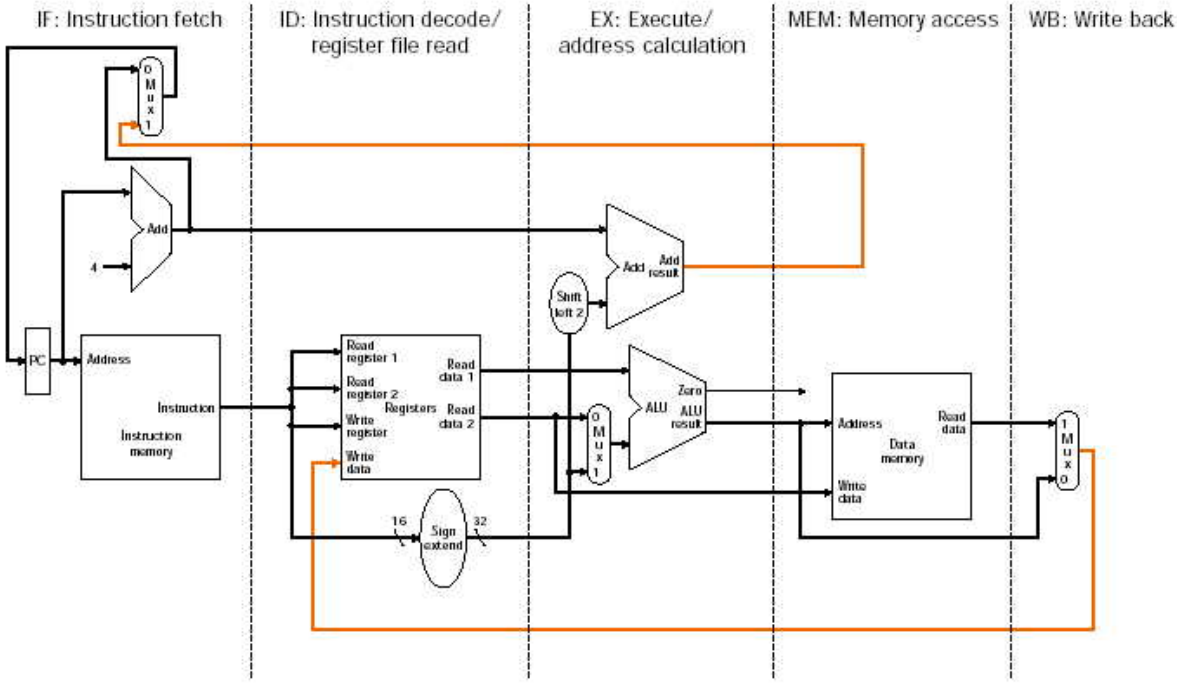

Figure 2: Single-cycle processor divided into 5 stages

path at this point allows us to build on the datapath introduced in the previous sections. In order to build the control path, we first have to break down which control signals are required, always referring to the control unit built with the single-cycle processor. Let us try to define the control signals required for our datapath. However, it is important to note that the pipeline registers are written at every clock cycle, hence do not require explicit write signals. That said, let us explore the other stages in the pipeline:

Instruction fetch and PC increment Asserted at every clock cycle, so no changes here;

Instruction decode and register file read Again, asserted at every clock cycle, so no changes here;

Execution and address calculation Need to select the result register, the ALU operation, and either read data 2 or the sign-extended immediate for the ALU;

Memory access Need to read from memory, write to memory or complete a branch; and

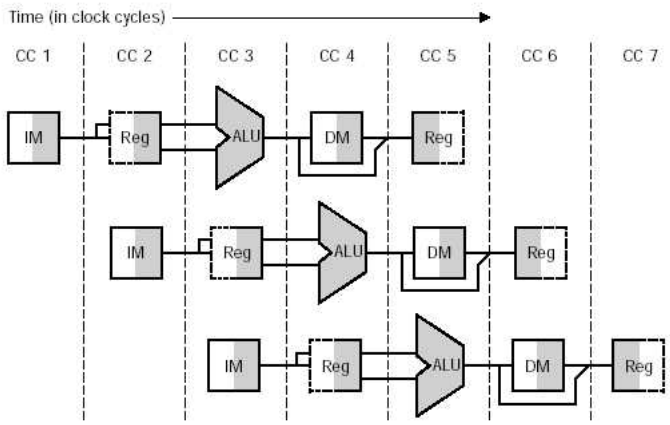

Figure 3: Multiple instructions running on the same datapath

Write back Need to send back either the ALU result or the memory value back to the register file.

Having already explored the control unit in the single-cycle processor, we replicate the final control truth table, with one major change. We group, in figure 5, the various control signals under the three major stages that need explicit control signals.

Adding the control unit to our datapath involves decoding the instruction in the ID stage, then passing on all control values to the subsequent pipeline registers. Figure 6 shows that process. Note that the control information has to be replicated down the pipeline, stripping the control signals for each stage when the latter is executing. The reason for this is that multiple instructions are executing at different stages at the same time, hence all control information has to be present at all times.

We finally group all the control information with our pipelined datapath to form the pipelined MIPS processor shown in figure 7. The reader is referred to the Acknowledgements section for further discussion on this diagram, including detailed functional examples.

4.2 Pipeline Hazards

There are three types of hazards that could occur in a pipelined execution of a processor. They are defined as:

Structural hazards Occur when a combination of instructions is not supported by the datapath, such as two stages of the pipeline requiring the same data unit;

Control hazards Occur when a decision is made based on the results of one

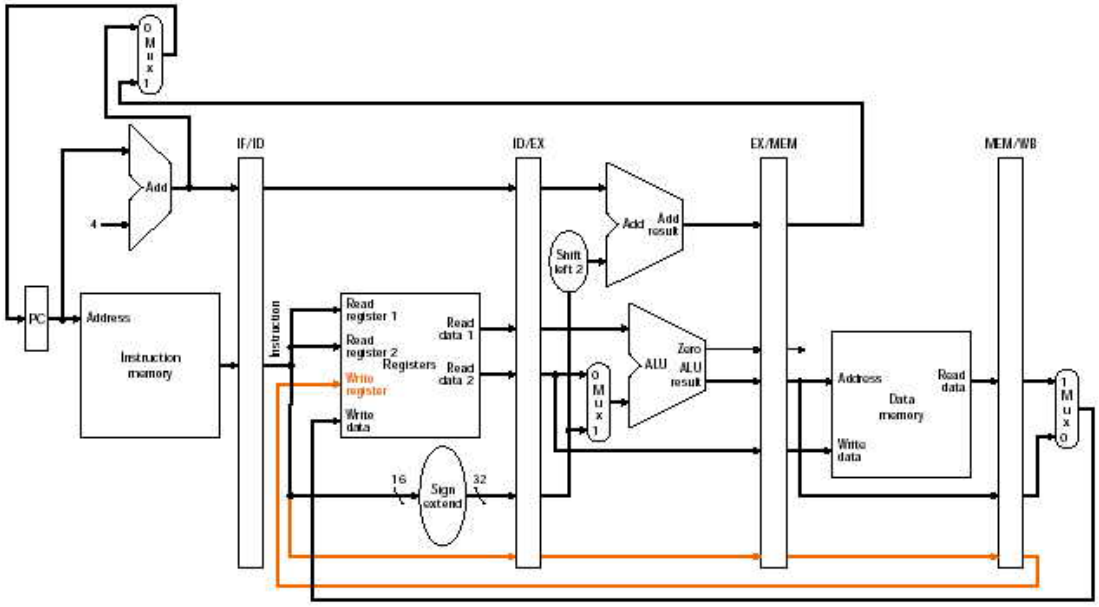

Figure 4: Pipelined MIPS processor

instruction, while others are executing, such as the decision whether a branch is taken or not; and

Data hazards Occur when an instruction depends on the results of an instruction resident on the pipeline, such as two R-type instructions linked by a source-destination register.

The pipeline will outright fail in the presence of structural hazards, hence they are not considered any further in this laboratory. Control and data hazards however can be resolved using techniques such as forwarding, predicting and stalling. There is no room to present all the theory for resolving both data and control hazards, but both have been clearly presented in the textbook and explained in the lecture notes. The reader is referred to the Acknowledgements section for a deeper discussion of the material. That said, the final datapath involved in this laboratory is presented in figure 8, where, in particular, a hazard detection unit, a forwarding unit and additional control signals have been added to the pipelined MIPS processor, in order to resolve both data and control hazards.

Instruction	Execution/Address Calculation stage control lines				Memory access stage control lines			stage control lines	
	Reg Dst	ALU Op1	ALU Op0	ALU Src	Branch	Mem Read	Mem Write	Reg write	Mem to Reg
R-format	1	1	0	0	0	0	0	1	0
lw	0	0	0	1	0	1	0	1	1
sw	X	0	0	1	0	0	1	0	X
beq	X	0	1	0	1	0	0	0	X

Figure 5: Control signal values

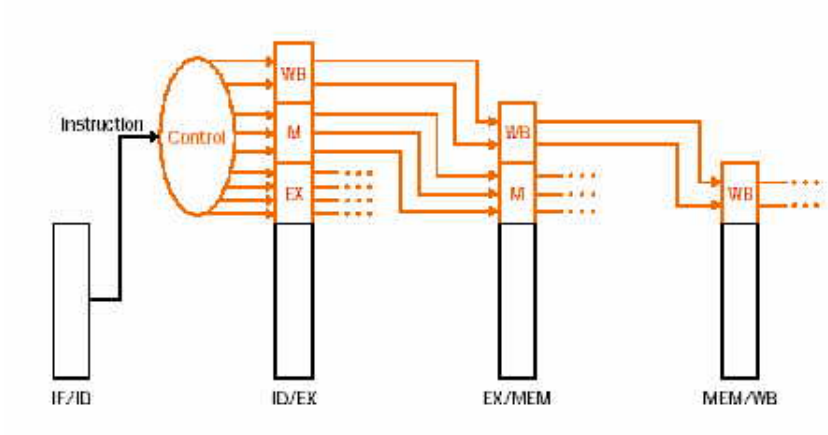

Figure 6: Control signal additions to pipeline registers

5 Laboratory

In this lab, you will proceed to implement a pipelined processor, with the datapath shown in figure 8. The input/output specification of the processor is shown in figure 9 and table 1. We will be again working with **8-bit datapaths** and **32-bit instruction widths**, while the control lines obviously remains constant. Also note that the 8-bit datapaths, will only allow for 8-bit number representations and operations. We will be implementing a register file of **eight 8-bit registers**. Instruction memory will remain at 32-bits, and will contain a maximum of 256 instructions. The data memory, however, will be 8-bits, and will contain a maximum of 256 words.

The MuxOut[7..0] and ValueSelect[2..0] output and input are intertwined in the manner shown in table 2.

The InstructionOut[31..0] and InstrSelect[2..0] output and inputs are intertwined in the manner shown in table 3, where any of the instructions being exe-

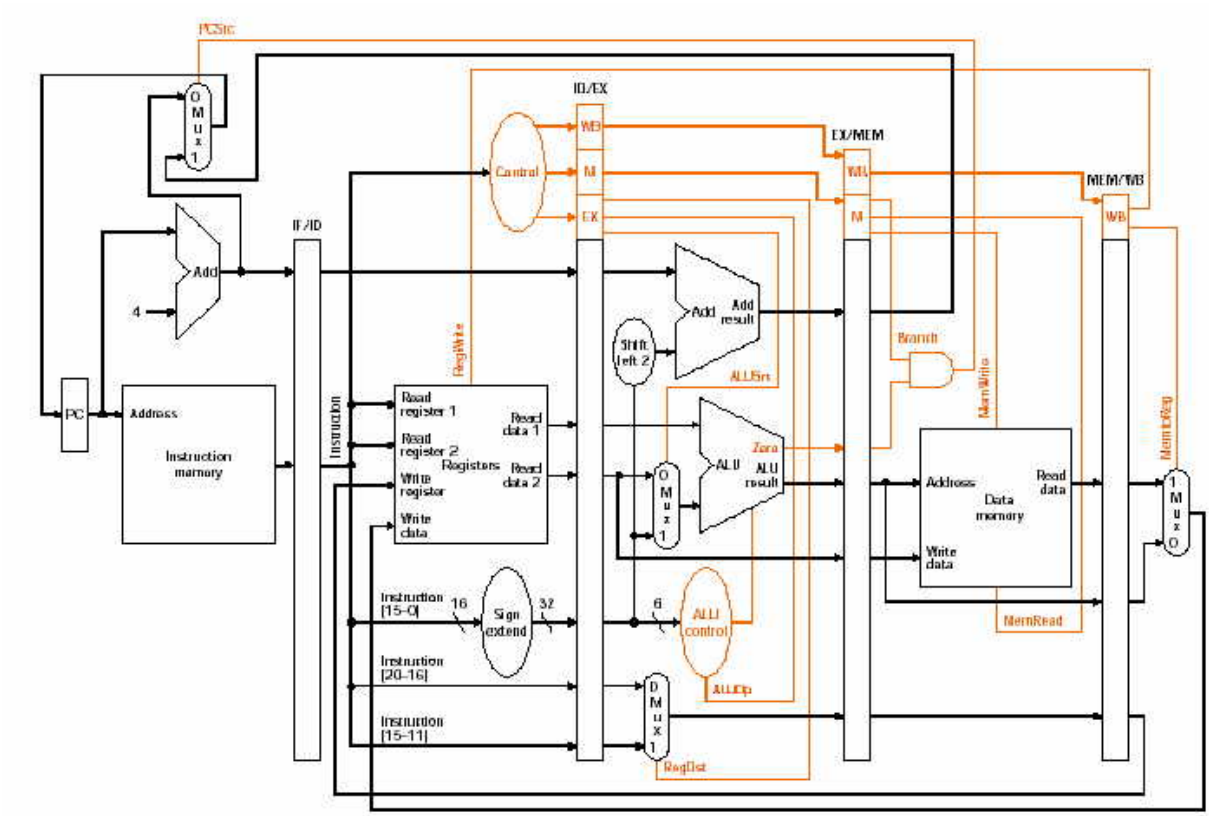

Figure 7: Pipelined MIPS processor with control path

cuted on the pipeline (i.e. InstructionOut1 to InstructionOut5) can be viewed.

5.1 Memory Blocks

Since we are operating with separate instruction and data memories, you will be allowed to use two Altera-supplied LPM cores. The instruction memory can be an instantiation of the LPM_ROM function (256x32), while the data memory can be an instantiation of the LPM_RAM_DQ function (256x32). That said, if the group chooses to implement their own RAM and ROM modules, they are welcome to do so.

A sample instruction memory initialization file (MIF) is given below:

```
DEPTH = 256;
WIDTH = 32;

ADDRESS_RADIX = HEX;
```

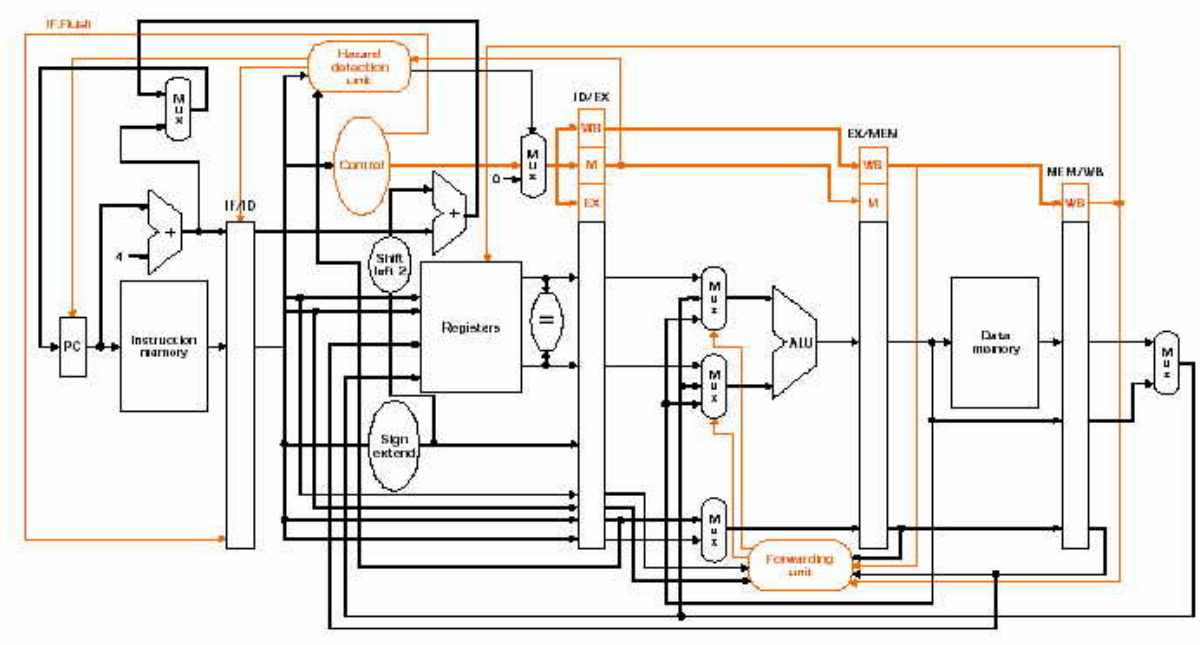

Figure 8: Pipelined MIPS processor with hazard resolution units

```
DATA_RADIX = BIN;

CONTENT
BEGIN

-- Use no operations (nop) for default instructions
[00..FF]:   00000000; -- nop(add $t1, $t1, $t1)

-- Place MIPS instruction here
00: 8C020000;   --lw $2,0 memory(00)=55
01: 8C030001;   --lw $3,1 memory(01)=AA
02: 00430820;   --add $1,$2,$3
03: AC010003;   --sw $1,3 memory(03)=FF
04: 1022FFFF;   --beq $1,$2,-4
05: 1021FFFA;   --beq $1,$1,-24

END;
```

A sample data memory initialization file (MIF) is given below:

```
DEPTH = 256;
WIDTH = 8;
```

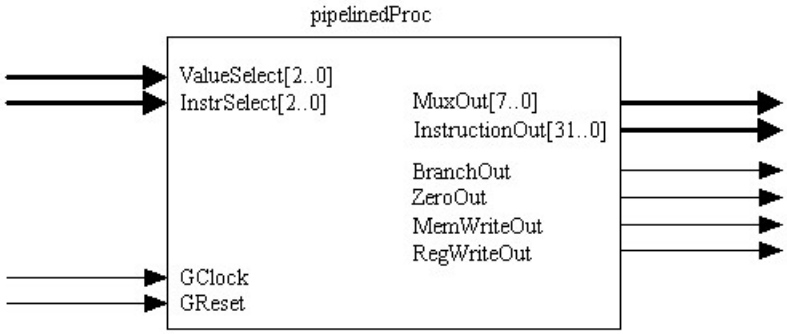

Figure 9: Pipelined processor system entity

Port Type	*Name*	*Description*
Input	GClock	Global clock needed to synchronize the circuitry
Input	GReset	Global reset needed to bring the internals to known states
Input	ValueSelect[2..0]	Selector for MuxOut[7..0]
Input	InstrSelect[2..0]	Selector for InstructionOut[31..0]
Output	MuxOut[7..0]	Multiplexer output controlled by ValueSelect[2..0]
Output	InstructionOut[31..0]	The current instruction being executed
Output	BranchOut	The branch control signal of each processor
Output	ZeroOut	The zero status signal of each processor
Output	MemWriteOut	The memory write control signal of each processor
Output	RegWriteOut	The register write control signal of each processor

Table 1: Input/Output Specification

```
ADDRESS_RADIX = HEX;
DATA_RADIX = HEX;

CONTENT
BEGIN

-- Default Memory Values
[00..FF]: 00;

-- Initial Values
00:   55;
01:   AA;
```

ValueSelect[2..0]	*MuxOut[7..0]*	*Description*
000	PC[7..0]	The program counter value
001	ALUResult[7..0]	The result of the current ALU operation
010	ReadData1[7..0]	The read data 1 port of the register file
011	ReadData2[7..0]	The read data 2 port of the register file
100	WriteData[7..0]	The write data port of the register file
Other	['0', RegDst, Jump, MemRead, MemtoReg, AluOp[1..0], AluSrc]	The remaining control information

Table 2: Output Multiplexer Selection

InstrSelect[2..0]	*InstructionOut[31..0]*
000	InstructionOut1[31..0]
001	InstructionOut2[31..0]
010	InstructionOut3[31..0]
011	InstructionOut4[31..0]
100	InstructionOut5[31..0]
Other	Zeros[31..0]

Table 3: Instruction Multiplexer Selection

```
END;
```

5.2 Verification: Running a Benchmark Program

Once completed, the design has to be tested and verified. The testing process can be accomplished throughout the design, with unit testing, then module testing, and finally system testing. As soon as the processor seems to be behaving according to specifications, a verification step is to be performed by running the following program, stored in instruction memory (remember to initialize the data memory as shown in the comments):

```
lw $2, 0;          $t2 = memory(00) = 55
lw $3, 1;          $t3 = memory(01) = AA
sub $1, $2, $3;    $t1 = $t2 - $t3 = 55
or $4, $1, $3;     $t4 = $t1 or $t3 = FF
%beq $1, $1, 20    test if $t1 = $t1 ?
sw $4, 3;          memory(03) = $t4 = FF
add $1, $2, $3;    $t1 = $t2 + $t3 = FF
sw $1, 4;          memory(04) = $t1 = FF
lw $2, 3;          $t2 = memory(03) = FF
lw $3, 4;          $t3 = memory(04) = FF
j 11;              jump to address 44
```

```
beq $1, $1, -44;  loop back to beginning of program
beq $1, $2, -8;   test if $t1 = $t2 ?
```

This program will have to be demonstrated as running to your TA as proof of success for your design. As well, the calculation of the maximum clock frequency has to be shown and explained in your report. For this, use the timing analyzer to determine the maximum clock rate of this pipelined processor, and verify correct operation at this clock rate in a simulation.

The above benchmark will distinguish between the pipelined and the single-cycle processors, but another benchmark will be used to test out the control hazards. For this benchmark, the group is asked to run the same previous benchmark, except for un-commenting (i.e. take out the % sign) the $beq\$1, \$1, 20$ instruction. Data hazards already occur in the original benchmark, so no modifications are required.

Finally, in order to measure the efficiency of our pipelined processor, we will need to calculate its CPU execution time as shown in the example on page 332 of our course book. Using that example, and the two benchmark programs above, calculate the CPU execution time of each of our processors for each program. Assume, as in the example, that memory units have a 200 ps operation time, while the register file has a 100 ps operation time. **However**, calculate the worst path delay of your ALU and adders, assuming that a gate delay is 0.01 ns. Show all your work and reasoning.

Compare your results with the ones found from the single-cycle processor laboratory, and state your reasoning why there are differing efficiencies between both designs.

5.3 Bonus: 32-bit Multiprocessor Design

For an **extra 5 marks**, modify your processor design to make it a completely 32-bit data and 32-bit instruction pipelined processor . Do not show all the components again, but instead, discuss the modifications performed on the components in order to run the 32-bit processors. If the datapath, control or instruction sequencing logic have changed, present and discuss the modifications. You have to demonstrate your new design to your TA.

5.4 Bonus: Exception Handling

For an **extra 5 marks**, implement the exception handling schemes shown in class and in the textbook (in particular, figure 4.67 on page 389). Note that only the arithmetic overflow exception has to be supported. Again, do not show all the components again, but instead, discuss the modifications performed on the components in order to realize the exception handling. If new components were added, it goes without saying that their design and realization has to be discussed. After completing the design, run a program where exceptions are purposely introduced and demonstrate its correct functionality to your TA.

5.5 Bonus: Board Demonstration

An **extra 10 marks** will be given to any group that actually performs live board demonstrations to the TA. This will include the Altera DE2-115 boards present in the lab and any re-design work that the group has to do to ensure correct design functionality. The components should remain the same, but depending on design methodologies and EDA tool support, some differences may have to be shown. After completing the design, run the benchmark program and demonstrate a working version to your TA.

6 Design Restrictions

- Verilog implementations will not be accepted. Perform all implementations in VHDL code only
- Behavioral level of modeling will not be accepted. Design should be done at the structural level of modeling
- Register Transfer Logic (RTL) design and coding will is mandatory
- Use graphical design for the top-level entity, and use your judgement for any other sub-blocks. However, all atomic modules have to be implemented in VHDL (i.e. D flip-flop, 1-bit adder, 1-bit comparator and so on)
- Designers can re-use modules that were designed in previous laboratories
- No core instantiations are allowed (i.e. LPMs from Altera or free IP cores from the Internet), except for the ones mentioned in the lab instructions (LPM_ROM and LPM_RAM_DQ). All remaining building blocks have to be designed and realized by the group
- The top level entity is given in input/output specification format, but the internals are left up to the group. A sample schematic solution for the single-cycle processor has been given, but the group is free to design the system using another methodology
- The design has to be synchronous and globally reset-able. This means that global clock and reset signals are required in all functional blocks
- Simulate all designs and check your simulation results with your theoretical ones (e.g. run the benchmark programs)
- If going for the bonus marks, download the design to the Cyclone chip on your DE-2 boards, and use the on-board DIP switches to inputs ValueSelect[2..0], and the sixteen LEDs to demonstrate correct output functionality (MuxOutput[7..0] and the control signals shown in figure 9)

- Only show the current instruction (InstructionOut1,2,3,4,5[31..0]) in simulation demonstration. You do not have to show it during the live demonstration
- Each group must demonstrate a working simulation of the laboratory to the TA, before the due time of the report

7 Report Reminders

- Include timing simulations with explanation for all VHDL source files
- Describe and comment all your VHDL source files
- Include a flowchart representation of your solution to the problem
- Include a block diagram of your solution to the problem
- If using ASM design, include all appropriate charts and paths (control and data)
- If using FSM design, include the state diagram with all appropriate inputs and outputs
- Describe, in your own words, your solution to the problem
- Describe your design obstacles and how they were overcome
- Append all VHDL source code and graphical design files to your report
- Submit a soft copy of all VHDL and graphical design files with your report
- Email a copy of all VHDL and graphical design files to your TA, following the rules posted on the web site in terms of naming conventions

8 Acknowledgements

All figures (except 9) are taken out of Chapter 4 of our reference textbook. For more detailed explanations and examples, please refer to *Computer Organization and Design* by David Patterson and John Hennessy (Morgan Kaufmann, 2009), as well as the course lecture notes.

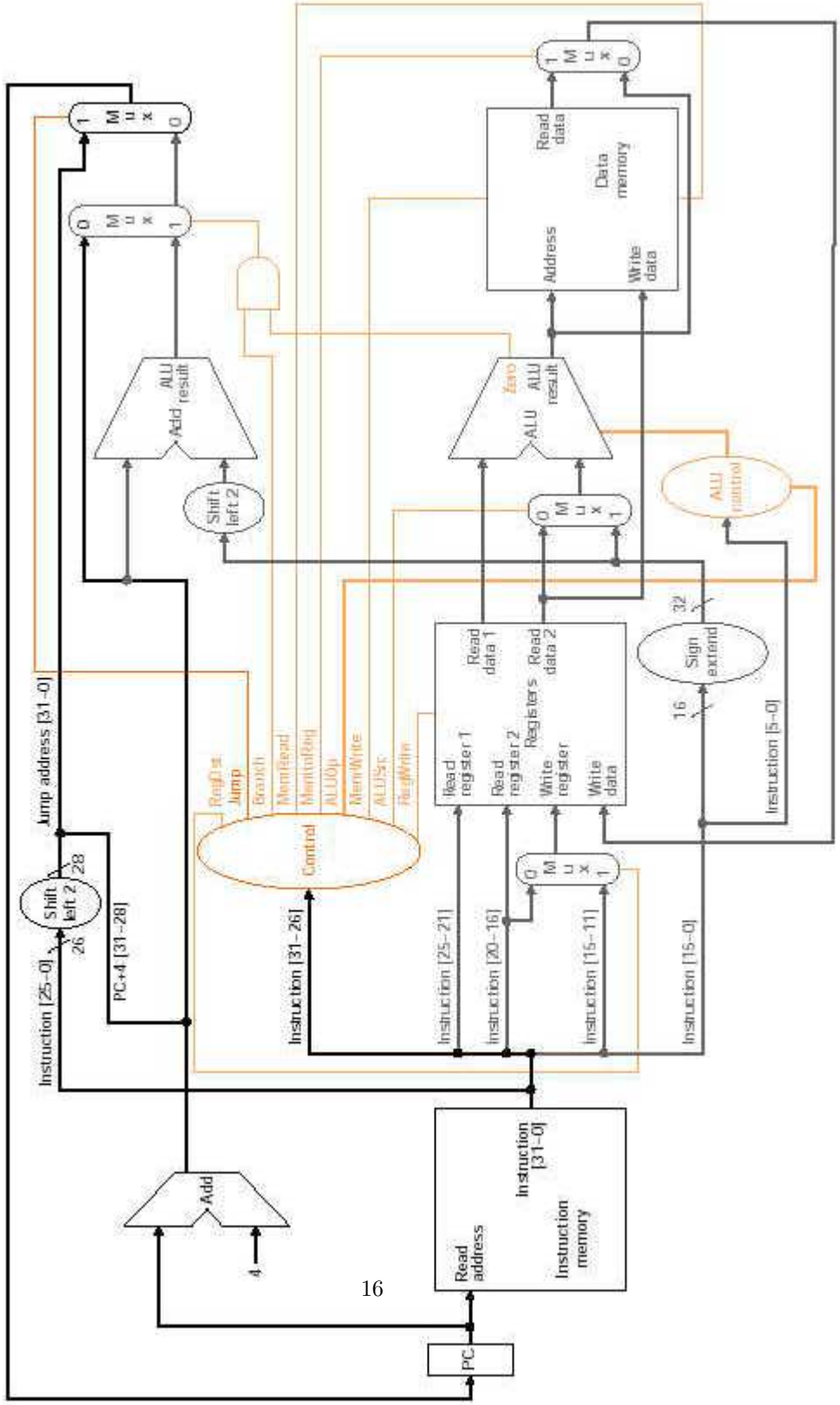

Figure 10: Single-cycle finalized RISC processor